

FieldMind

An AI-assisted research-farm manager for inventory, treatment planning, and stockout prediction at the plot level.

AUTHOR	Ishrat Jandu
DEPARTMENT	Biological Systems Engineering, University of Nebraska–Lincoln
COURSE	AGST 492 — Agentic AI for Workflow Automation
REPOSITORY	github.com/Ishrat2110/fieldmind
SUBMISSION	May 2026

ABSTRACT

FieldMind is a working web application for managing the day-to-day inputs of a research farm: what you have on the shelf, where it is going, when you applied it, and when you will run out. The submitted version covers a seeded scenario based on the UNL East Campus research farm: 2 fields, 16 plots split between corn and soybean, 7 inventory items, 40 planned treatments, and a small fleet of equipment shared across plots.

Two pieces make it more than a CRUD app. The first is a Gemini-backed reorder engine that watches the inventory table and writes a plain-English suggestion the moment an item drops past its threshold, with a rule-based fallback so the app keeps working when the API key is absent. The second is a set of Streamlit visualization tools (a clickable plot map, a depletion-forecast chart, and a shapefile analyzer) that a farm manager could realistically open beside the main dashboard during a planning meeting.

1. Motivation

Research farms run on spreadsheets. The herbicide inventory I worked with at the start of the semester was a single Excel file passed between the farm manager and a few graduate students, with no link between what was on the shelf and what got sprayed on which plot. When something ran out, nobody noticed until someone showed up to use it. The goal of FieldMind was to take the same data the spreadsheet already tracked and put it behind a web app that could answer two questions in real time: *what is on hand* and *when will it run out*.

2. What the application does

Inventory

Fertilizers, herbicides, fungicides, fuel, and seed. Reorder threshold, supplier, unit cost, and a 7-day consumption sparkline per item. Below-threshold items surface as red navigation badges.

Treatment planning

Plans scheduled per plot and tied to a crop growth stage. Overdue and upcoming plans surface on the dashboard.

Usage logging

A guided form (item, plot, equipment, quantity) that deducts from inventory in the same transaction. Filterable history with CSV export.

AI reorder suggestions

The engine in `reorder_ai.py` calls Gemini with a structured payload (current stock, recent daily burn, lead time) and parses a JSON response with quantity, target date, and a one-line justification. If the key is missing or the call fails, a deterministic rule takes over.

Maps and GIS

Streamlit `farm_map.py` draws plots in Plotly with hover details and a per-item depletion-forecast curve. `shapefile_analyzer.py` reads real shapefiles, colors polygons by attribute, and matches them back to database plots.

Authentication

UUID-based login, signed Flask sessions, Flask-WTF CSRF on every form, and three roles (admin, manager, viewer) checked per request.

3. Architecture

The web app is a single Flask service (`server.py`) that owns routes, templates, and forms, talking to SQLite through SQLAlchemy 2.x. The reorder logic lives in its own module so the request handler does not have to know how a suggestion is generated; `server.py` calls `maybe_generate_suggestion(item, session)` after every inventory change and the engine decides AI vs. rule-based. Streamlit tools share the same database and models but run as separate processes, so a problem in one cannot take the dashboard down.

4. Database schema

Fourteen tables across four domains, defined in `models.py`:

Users & Access	Crops & Land	Inputs & Equipment	Tracking
users farms farm_members	crop_species crop_varieties growth_stages fields plots	inventory_items equipment	usage_logs treatment_plans activity_logs notifications

5. Tech stack

LAYER	CHOICE	REASON
Web framework	Flask + Jinja2	Server-rendered pages were the right call for a single-tenant internal tool. No SPA churn.
Forms / CSRF	Flask-WTF	Token-on-every-form pattern; simpler than wiring CSRF by hand.
Database	SQLite via SQLAlchemy 2.x	One file, easy to seed, easy to ship for a class submission. Postgres migration is a one-line URL change.
AI engine	Google Gemini (google-gemini)	Free tier covers development; structured JSON output mode keeps the response parseable.
Visualization	Plotly + Streamlit	Streamlit gave me three working analyst tools without writing a frontend for each one.
GIS	Geopandas + Shapely + pygrio	Standard stack for reading shapefiles in Python, no QGIS dependency.
Frontend	Tailwind CDN + vanilla JS	Custom Mariner-blue / Cornhusker-red palette derived from UNL brand, no Bootstrap.

6. AI integration approach

Two design choices shaped how the model is used. First, the AI never writes to the database directly. It returns a *suggestion*, the suggestion is stored as a `notifications` row, and a human has to confirm it before any reorder is recorded. That keeps the model out of any path where a bad response could corrupt inventory state. Second, the reorder engine ships with three caps (`REORDER_AI_CACHE_HOURS`, `REORDER_AI_DEFAULT_LEAD_DAYS`, `REORDER_AI_MAX_QTY_MULTIPLIER`) read from the environment, so the same code can be tuned for a different farm without a redeploy.

7. What I would change next

- **Move off SQLite for multi-user writes.** Concurrent writes from two laptops will hit locks; Postgres is the obvious next step.
- **Tighter feedback loop on the AI.** The model does not see whether the previous suggestion was accepted or ignored. Storing the user's response on the notification row and feeding the last few outcomes back into the next prompt would let suggestions improve over a season.
- **Mobile capture.** The logging form works on a phone but is not optimized for one-handed use in a field. A small PWA capture flow that posts to the same endpoint is a clear win.
- **Treatment recommendation.** The treatment table records what was planned. The next jump is to suggest *what* to apply at a given growth stage from past records.

8. Outcomes

The submitted version runs end-to-end against a clean clone in under five minutes (`pip install`, `python database.py`, `python server.py`), with live AI suggestions when a Gemini key is set. Roughly 1,700 lines of Python plus templates, a small pytest suite, and a smoke-test script in `tests/`. The public README documents install, environment variables, default credentials, and file layout so a grader can pick it up without me sitting next to them.